

Experiment 1

```
import re

text = input("Enter the text: ")
pattern = input("Enter the pattern: ")

matches = re.findall(pattern, text)

if matches:
    print("Matched Patterns:", matches)
else:
    print("No match found")
```

```
Enter the text: Tharunya
Enter the pattern: name
No match found
```

Experiment 2

```
def finite_state_automaton(string):
    if string.endswith("ab"):
        return "Accepted"
    else:
        return "Rejected"

text = input("Enter a string: ")

result = finite_state_automaton(text)

print(result)
```

```
Enter a string: aaab
Accepted
```

Experiment 3

```
import nltk
from nltk.tokenize import word_tokenize
from nltk.stem import PorterStemmer

# Download required resources (run once)
nltk.download('punkt')
nltk.download('punkt_tab')

ps = PorterStemmer()

text = input("Enter a sentence: ")

words = word_tokenize(text)

print("Morphological Analysis:")

for word in words:
    print(word, "->", ps.stem(word))
```

```
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt_tab.zip.
Enter a sentence: the boys are playing
Morphological Analysis:
the -> the
boys -> boy
are -> are
playing -> play
```

Experiment 4

```
def plural(noun):

    if noun.endswith(("s", "x", "z", "ch", "sh")):
        return noun + "es"
```

```
elif noun.endswith("y") and noun[-2].lower() not in "aeiou":
    return noun[:-1] + "ies"

else:
    return noun + "s"

word = input("Enter a singular noun: ")

print("Plural Form:", plural(word))
```

Enter a singular noun: Book
Plural Form: Books

Experiment 5

```
from nltk.stem import PorterStemmer

# Create Porter Stemmer object
ps = PorterStemmer()

# List of words
words = ["playing", "running", "happiness", "studies", "connected"]

print("Original Word\tStemmed Word")
print("-----")

for word in words:
    print(word, "\t", ps.stem(word))
```

Original Word	Stemmed Word
-----	-----
playing	play
running	run
happiness	happi
studies	studi
connected	connect

Experiment 6

```
import nltk
from nltk.tokenize import word_tokenize
from nltk.util import bigrams

# Download required resource (run once)
nltk.download('punkt')

text = input("Enter a sentence: ")

words = word_tokenize(text)

bigram_list = list(bigrams(words))

print("Generated Bigrams:")

for bg in bigram_list:
    print(bg)
```

[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data] Package punkt is already up-to-date!
Enter a sentence: Natural language processing
Generated Bigrams:
('Natural', 'language')
('language', 'processing')

Experiment 7

```
import nltk
from nltk.tokenize import word_tokenize

# Download required resources (Run once)
nltk.download('punkt')
nltk.download('averaged_perceptron_tagger_eng')

text = input("Enter a sentence: ")

words = word_tokenize(text)
```

```
tags = nltk.pos_tag(words)
```

```
print("POS Tagged Words:")
for word, tag in tags:
    print(word, ":", tag)
```

```
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package averaged_perceptron_tagger_eng to
[nltk_data]   /root/nltk_data...
[nltk_data]   Unzipping taggers/averaged_perceptron_tagger_eng.zip.
Enter a sentence: The boy is playing football
POS Tagged Words:
The : DT
boy : NN
is : VBZ
playing : VBG
football : NN
```

Experiment 8

```
# Simple probabilistic dictionary
pos_dict = {
    "the": "DT",
    "boy": "NN",
    "girl": "NN",
    "plays": "VBZ",
    "is": "VBZ",
    "playing": "VBG",
    "football": "NN",
    "runs": "VBZ",
    "quickly": "RB"
}

sentence = input("Enter a sentence: ").lower()

words = sentence.split()

print("Stochastic POS Tagging:")

for word in words:
    tag = pos_dict.get(word, "UNK")
    print(word, ":", tag)
```

```
Enter a sentence: The boy is playing football
Stochastic POS Tagging:
the : DT
boy : NN
is : VBZ
playing : VBG
football : NN
```

Experiment 9

```
import re

sentence = input("Enter a sentence: ")

words = sentence.split()

print("Rule-Based POS Tagging:")

for word in words:
    if re.fullmatch(r"*.ing", word):
        tag = "VBG"

    elif re.fullmatch(r"*.ed", word):
        tag = "VBD"

    elif re.fullmatch(r"*.ly", word):
        tag = "RB"

    elif word.lower() in ["the", "a", "an"]:
        tag = "DT"

    else:
        tag = "NN"
```

```
print(word, ":", tag)
```

Enter a sentence: Natural language processing
Rule-Based POS Tagging:
Natural : NN
language : NN
processing : VBG

Experiment 10

```
# Initial tagged sentence
tagged_sentence = [
    ("The", "DT"),
    ("boy", "NN"),
    ("is", "NN"),
    ("playing", "NN")
]

print("Before Transformation:")
for word, tag in tagged_sentence:
    print(word, ":", tag)

print("\nAfter Transformation:")

new_tags = []

for word, tag in tagged_sentence:

    if word == "is":
        tag = "VBZ"

    elif word.endswith("ing"):
        tag = "VBG"

    new_tags.append((word, tag))

for word, tag in new_tags:
    print(word, ":", tag)
```

Before Transformation:
The : DT
boy : NN
is : NN
playing : NN

After Transformation:
The : DT
boy : NN
is : VBZ
playing : VBG